

Zadanie 11

Natalia Włódyka

Grudzień 27, 2025

1 Polecenie zadania

Celem zadania jest skonstruowanie naturalnego splajnu kubicznego dla funkcji i węzłów z zadania 09.

2 Zadane węzły i funkcja

Zadana w zadaniu 09 funkcja dla $x \in [-1.25, 1.25]$:

$$f(x) = \frac{1}{1 + 5x^2}$$

Węzły Czebyszewa:

$$x_j = \cos\left(\frac{2j-1}{16}\pi\right), \quad j = 1, 2, \dots, 8$$

3 Układ równian dla niewiadomych ξ_j

Zauważmy, że gdy odległość między węzłami nie jest stała $h_j = x_{j+1} - x_j$, otrzymujemy wtedy poniższy układ trójdzielny równań dla niewiadomych ξ :

$$\begin{bmatrix} 2(h_0 + h_1) & h_1 & 0 & \dots & 0 \\ h_1 & 2(h_1 + h_2) & h_2 & \dots & \dots \\ 0 & h_2 & 2(h_2 + h_3) & \dots & 0 \\ \dots & \dots & \dots & \dots & h_{n-2} \\ 0 & \dots & 0 & h_{n-2} & 2(h_{n-2} + h_{n-1}) \end{bmatrix} \begin{bmatrix} \xi_1 \\ \xi_2 \\ \xi_3 \\ \vdots \\ \xi_{n-1} \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{bmatrix} \quad (1)$$

Gdzie:

$$b_j = 6\left(\frac{f_{j+1} - f_j}{h_j} - \frac{f_j - f_{j-1}}{h_{j-1}}\right), \quad j = 1, 2, \dots, n-2$$

Powyższy układ rozwiążemy za pomocą algorytmu Thomasa.

4 Wyznaczanie współczynników splajnu i jego wartości

Wartości splajnu dla dowolnego $x \in [x_j, x_{j+1}]$ wyliczamy ze wzoru:

$$P(x) = Af_j + Bf_{j+1} + C\xi_j + D\xi_{j+1}$$

Gdzie:

$$A = \frac{x_{j+1} - x}{x_{j+1} - x_j}, B = \frac{x - x_j}{x_{j+1} - x_j}, C = \frac{1}{6}(A^3 - A)(x_{j+1} - x_j)^2, D = \frac{1}{6}(B^3 - B)(x_{j+1} - x_j)^2$$

Dla uproszczenia programu za $x_{j+1} - x_j$ wstawiamy zmienną h .

5 Kod w Pythonie

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from numpy.typing import NDArray
4
5 vector = NDArray[np.float64]
6 matrix = NDArray[np.float64]
7 number = np.float64
8
9 def thomas_method(sub: vector, diagonal: vector, sup: vector, b:
    vector) -> vector:
10     N = len(sub)
11     cp = np.zeros(N, dtype= number)
12     dp = np.zeros(N, dtype= number)
13     xi = np.zeros(N, dtype= number)
14
15     #Forward
16     cp[0] = sup[0] / diagonal[0]
17     dp[0] = b[0] / diagonal[0]
18     for i in np.arange(1, (N), 1):
19         dnum = diagonal[i] - sub[i] * cp[i - 1]
20         cp[i] = sup[i] / dnum
21         dp[i] = (b[i] - sub[i] * dp[i - 1]) / dnum
22
23     #Back
24     xi[(N - 1)] = dp[N - 1]
25
26     for i in np.arange((N - 2), -1, -1): #
27         xi[i] = (dp[i]) - (cp[i]) * (xi[i + 1])
28
29     return xi
30
31 def f_x(x: number) -> number:
32     return 1.0 / (1 + 5 * x * x)
33
34 def x_j(j: int) -> number:
35     return -np.cos(((2*j - 1) / 16)* np.pi)
36
37 def spline_coefficients(x_val: number, x_j: number, x_j1: number)
    -> tuple[number, number, number, number]:
```

```

38     h = x_j1 - x_j
39     A = (x_j1 - x_val) / h
40     B = (x_val - x_j) / h
41     C = (1.0 / 6.0) * (A ** 3 - A) * (h**2)
42     D = (1.0 / 6.0) * (B ** 3 - B) * (h**2)
43
44     return A, B, C, D
45
46 def spline_function(f_x: vector, x_val : number, x:vector, xi:
47     vector) -> number:
48     j = np.searchsorted(x, x_val) - 1
49     if j < 0:
50         j = 0
51     if j >= len(x) - 1:
52         j = len(x) - 2
53
54     A, B, C, D = spline_coefficients(x_val, x[j], x[j + 1])
55
56     P_x = A * f_x[j] + B * f_x[j + 1] + C * xi[j] + D * xi[j + 1]
57     return P_x
58
59 def main():
60     n = 8
61     x: vector = np.zeros(n, dtype=number)
62     for i in range(n):
63         x[i] = x_j(i + 1)
64
65     f: vector = np.zeros(n, dtype=number)
66
67     for i in range(n):
68         f[i] = f_x(x[i])
69
70     h = np.diff(x)
71     diagonal: vector = np.zeros(n - 2, dtype=number)
72     subdiagonal: vector = np.zeros(n - 3, dtype=number)
73     superdiagonal: vector = np.zeros(n - 3, dtype=number)
74     b: vector = np.zeros(n - 2, dtype=number)
75
76     #Tworzymy macierz
77     for i in range(n - 3):
78         diagonal[i] = 2 * (h[i] + h[i + 1])
79         subdiagonal[i] = h[i + 1]
80         superdiagonal[i] = h[i + 1]
81         b[i] = 6 * ((f[i + 2] - f[i + 1]) / h[i + 1] - (f[i + 1] -
82             f[i]) / h[i])
83
84     vector_xi_0 = thomas_method(subdiagonal, diagonal,
85         superdiagonal, b)
86     vector_xi: vector = np.zeros(n, dtype=number)
87     vector_xi[1:n-2] = vector_xi_0
88
89     #Tworzymy wykres
90     range_of_x = np.linspace(x[0], x[-1], 1000)
91     spline = np.zeros_like(range_of_x)
92     for k, val in enumerate(range_of_x):
93         spline[k] = spline_function(f,val, x, vector_xi)

```

```

92     plt.plot(range_of_x, spline, label='Naturalny splajn kubiczny',
93              color='purple')
94     plt.plot(x, f, 'o', label='Węzły interpolacyjne', color='pink')
95     plt.title('Naturalny splajn kubiczny')
96     plt.grid(True)
97     plt.xlabel('x')
98     plt.ylabel('y')
99     plt.legend()
100    plt.show()
101
102 if __name__ == "__main__":
103     main()

```

6 Wykresy naturalnego splajnu kubicznego

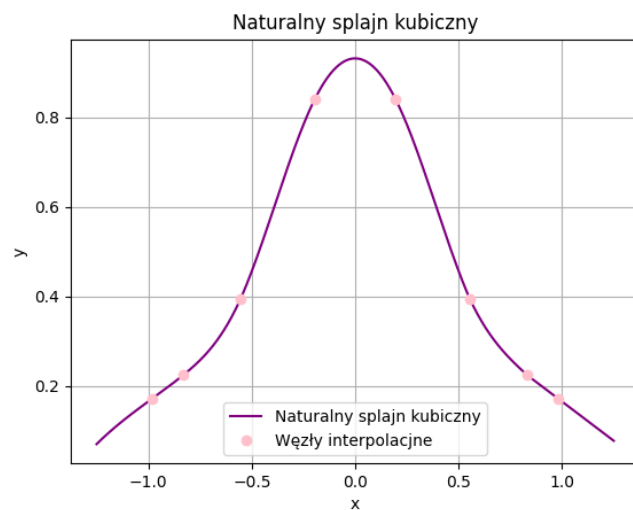


Figure 1: Splajn w przedziale $x \in [-1.25, 1.25]$

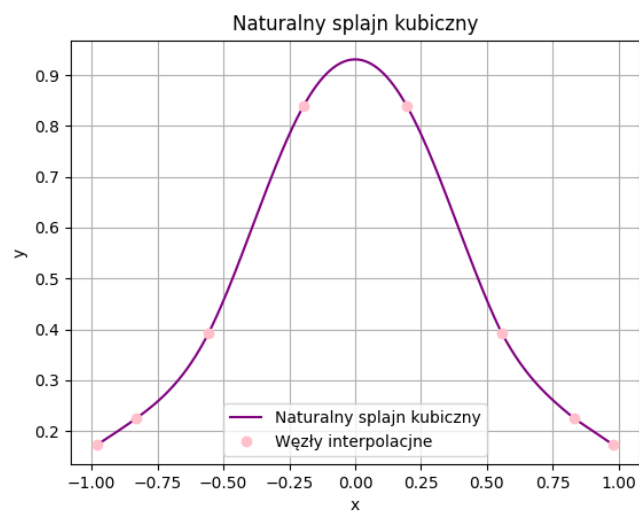


Figure 2: Splajn w przedziale $x \in [-1.0, 1.0]$